

Documentacion Tecnica

Impacto Espacial

Shoot 'em up lateral 2D | Unreal Engine 4.27 | C++

Este documento explica COMO funciona el juego de principio a fin: su arquitectura, el flujo de la partida, los 8 patrones de diseno implementados (con su mecanismo), la documentacion de todas las clases, las decisiones tecnicas y un diccionario de la sintaxis de Unreal y C++ usada.

Contenido

- 1. Descripcion del juego
- 2. Arquitectura y subsistemas
- 3. Flujo de la partida (game loop)
- 4. Patrones de diseno (los 8)
- 5. Como colaboran los patrones
- 6. Documentacion de clases
- 7. Decisiones tecnicas y correccion de crashes
- 8. Diccionario de sintaxis (Unreal + C++)
- 9. Como compilar y ejecutar

1. Descripcion del juego

Impacto Espacial es un **shoot 'em up de desplazamiento lateral** en 2D (el juego ocurre en el plano X-Z; el eje Y esta bloqueado). El jugador controla una nave que se mueve y dispara; los enemigos llegan desde la derecha con distintos patrones de movimiento.

Hay **3 niveles**, cada uno termina con un JEFE: **Dragon** (nivel 1), **PezMonstruo** (nivel 2) y **Kraken** (nivel 3, jefe final). Al vencer al Kraken se muestra la pantalla de Victoria; si la nave pierde sus 3 vidas, aparece Game Over.

Mecanicas principales

- Movimiento de la nave y disparo continuo (con piscina de balas).
- Habilidades especiales: misil oscilante, laser recto y laser vertical (teclas 1, 2, 3).
- Power-up (capsula) que mejora el arma con disparo en abanico.
- Oleadas de enemigos generadas automaticamente; al limpiar la oleada aparece el jefe.
- HUD con vida, vidas y puntaje; menus de inicio, victoria y game over.
- Escenario (fondo de estrellas y tunel) generado por codigo.

2. Arquitectura y subsistemas

El codigo (modulo **ImpactoEspacial**) se organiza en carpetas por responsabilidad:

Subsistema	Que contiene
Nucleo	GameManager (Singleton + State), GameInstance, GameMode, FachadaJuego (Facade), EstadoJuego (State), MapaEstelar (escenario).
Jugador	NaveJugador: el Pawn del jugador. Object Pool de balas, Subject del Observer y portador del arma (Decorator).
Enemigos	EnemigoBase (contexto de Strategy), JefeBase y jefes (Dragon, PezMonstruo, Kraken), FabricaEnemigos (Factory), ConstructorEnemigo (Builder), GeneradorEnemigos y las Estrategias de movimiento.
Proyectiles	ProyectilBase (elemento del pool) y variantes: Laser, LaserVertical, Misil.
Armas	Arma (componente del Decorator) y DecoradorArma (decoradores que envuelven el arma).
PowerUps	CapsulaPoder: mejora el arma de la nave al recogerla.
UI	PantallaHUD (Observer), PantallaMenu, PantallaGameOver, PantallaVictoria.

3. Flujo de la partida (game loop)

Asi encaja todo durante una partida, paso a paso:

1. El **GameMode** arranca el nivel: muestra el menu (si esta asignado) y crea el escenario (MapaEstelar).
2. Desde el menu, el boton Jugar carga el nivel de juego (OpenLevel).
3. La **NaveJugador** arranca (BeginPlay): llama a la **Facade** IniciarPartida (reinicia el GameManager, pone nivel 1 y activa el generador), crea el HUD y lo SUSCRIBE a sus eventos

(Observer), llena la piscina de balas (Object Pool) y se equipa el arma basica (Decorator).

4. El **GeneradorEnemigos**, por temporizador, pide enemigos a la **Fabrica**, que los arma con el **Builder** y les asigna una **Estrategia** de movimiento.

5. El jugador dispara: el **arma actual** (Decorator) decide como (1 bala, o en abanico si esta envuelta por un power-up) usando la piscina.

6. Al destruir un enemigo, **EnemigoBase** suma puntos y avisa al **GameManager**; la nave **PUBLICA** el cambio y el HUD se actualiza solo (Observer).

7. Cuando se acaban los enemigos de la oleada, el generador pide el JEFE del nivel a la Fabrica.

8. Al vencer al jefe se pasa al siguiente nivel; tras el jefe del nivel 3 se muestra Victoria.

9. Si la nave pierde las 3 vidas, la Facade ejecuta **TerminarPartida** (estado **GameOver** + pantalla).

10. El **GameManager** mantiene el estado del juego como objetos (State): Menu -> Jugando -> (Pausado) -> **GameOver** / Victoria.

4. Patrones de diseño (los 8)

1. Strategy (Estrategia)

Que es: Define una familia de algoritmos intercambiables y los encapsula, para cambiar el comportamiento sin tocar la clase que los usa.

En el juego: El movimiento de los enemigos: cada uno tiene una estrategia (Lineal, ZigZag, Diagonal, Perseguidor).

Como funciona: EnemigoBase guarda un puntero a la interfaz **UEstrategiaMovimiento** y en su **Tick** le pide la nueva posición. La estrategia se puede cambiar con **EstablecerEstrategia()**.

Mecanismo clave (lo que define el patron): Interfaz abstracta con método **PURE_VIRTUAL CalcularNuevaPosicion()** + delegación desde el contexto.

Archivos: EstrategiaMovimiento.h y sus 4 hijas; EnemigoBase.h/.cpp.

2. Factory (Fabrica)

Que es: Centraliza la creación de objetos en un solo lugar, ocultando que clase concreta se instancia.

En el juego: Creación de enemigos y jefes: nadie hace **SpawnActor** de un tipo concreto; se lo pide a la fábrica.

Como funciona: **UFabricaEnemigos::GenerarEnemigo(tipo)** y **GenerarJefe(nivel)** devuelven el producto ya configurado. El generador solo pide el tipo.

Mecanismo clave (lo que define el patron): Métodos creadores que devuelven el producto por su tipo abstracto (**AEnemigoBase / AJefeBase**). Es un Simple Factory.

Archivos: FabricaEnemigos.h/.cpp.

3. Singleton

Que es: Garantiza que exista UNA sola instancia de una clase, con un punto de acceso global.

En el juego: El GameManager: el 'cerebro' del juego (puntuación, nivel, oleadas, estado).

Como funciona: Constructor privado e instancia estática única. **ObtenerInstancia()** la crea una sola vez (en el paquete transitorio, protegida con **AddToRoot()**) y devuelve siempre la misma; así es segura entre niveles.

Mecanismo clave (lo que define el patron): Constructor privado + miembro **static Instancia** + acceso único **ObtenerInstancia()**.

Archivos: GameManager.h/.cpp.

4. Observer (Observador)

Que es: Un objeto (Sujeto) avisa a otros (Observadores) cuando cambia, sin conocerlos directamente.

En el juego: La nave (Sujeto) avisa al HUD (Observador) cuando cambian vida, vidas o puntos.

Como funciona: La nave declara eventos (delegados) y los dispara con `Broadcast()` (el 'publish'). El HUD se suscribe con `AddDynamic()` y reacciona solo.

Mecanismo clave (lo que define el patron): `OnSaludCambiada.Broadcast(...)` (publish) + `AddDynamic(this, &Func)` (subscribe).

Archivos: NaveJugador.h/.cpp (Sujeto); PantallaHUD.h/.cpp (Observador).

5. Builder (Constructor)

Que es: Construye un objeto complejo PASO A PASO con metodos encadenados.

En el juego: Armar cada enemigo (vida, velocidad, estrategia, malla, escala). La Fabrica lo usa por dentro.

Como funciona: `FConstructorEnemigo().ConVida(100).ConVelocidad(400).ConEstrategia(...).ConMalla(...).Construir(mundo, pos)`.

Mecanismo clave (lo que define el patron): Metodos `ConX()` que devuelven `*this` (encadenables) + un `Construir()` final que entrega el objeto.

Archivos: ConstructorEnemigo.h/.cpp; lo usa FabricaEnemigos.cpp.

6. Facade (Fachada)

Que es: Ofrece una interfaz simple que esconde la coordinacion de varios subsistemas.

En el juego: Operaciones de alto nivel del juego (iniciar/terminar partida, mostrar victoria).

Como funciona: `UFachadaJuego::TerminarPartida()` hace, en una sola llamada: cambiar estado a GameOver, mostrar el widget y liberar el cursor. La nave ya no orquesta todo a mano.

Mecanismo clave (lo que define el patron): Clase con metodos de alto nivel que por dentro llaman a GameManager + Generador + UI + PlayerController.

Archivos: FachadaJuego.h/.cpp; la usa NaveJugador.cpp.

7. State (Estado)

Que es: Cada estado es una CLASE con su propio comportamiento; el objeto delega en el estado actual y cambia entre ellos.

En el juego: El estado del juego: Menu, Jugando, Pausado, GameOver, Victoria.

Como funciona: El GameManager guarda un `UEstadoJuego* EstadoObjeto` y delega en el. `PausarJuego()` llama a `EstadoObjeto->Pausar(this)`: solo EstadoJugando responde; los demas ignoran. `CambiarEstado()` intercambia el objeto.

Mecanismo clave (lo que define el patron): Clases de estado con `Entrar()/Pausar()/Reanudar()` + delegacion + `CambiarEstado()`.

Archivos: EstadoJuego.h/.cpp; integrado en GameManager.

8. Decorator (Decorador)

Que es: Envuelve un objeto para anadirle funciones sin modificar su clase, y se pueden APILAR.

En el juego: Los power-ups del arma: un arma basica se envuelve con efectos (disparo triple).

Como funciona: La nave tiene un `UArma* ArmaActual` (empieza basica). Al recoger la capsula, se envuelve con `UDecoradorDisparoTriple`; al agotarse, se desenvuelve. El decorador llama al arma envuelta Y ADEMÁS anade lo suyo.

Mecanismo clave (lo que define el patron): `UDecoradorArma::ArmaEnvuelta + Disparar()` que delega en el envuelto y agrega su efecto (apilable).

Archivos: `Arma.h/.cpp` y `DecoradorArma.h/.cpp`; integrado en `NaveJugador`.

5. Como colaboran los patrones

Los patrones no estan aislados: trabajan juntos durante la partida.

- La **Factory** usa el **Builder** para armar cada enemigo.
- El **Builder** crea la **Strategy** de movimiento del enemigo.
- La **Facade** usa el **Singleton** (GameManager) por dentro.
- El **Singleton** contiene el **State** (su estado es un objeto, no un enum).
- El **Decorator** (arma) usa el **Object Pool** para lanzar las balas.
- El **Observer** conecta la nave (datos) con el HUD (presentacion) sin acoplarlos.

6. Documentacion de clases

Nucleo

Clase	Que hace
UGameManager	Singleton: cerebro del juego (puntos, nivel, oleadas, estado). Contiene el State.
USpaceImpactGameInstance	Persiste datos entre niveles (puntuacion final, nivel alcanzado).
AImpactoEspacialGameModeBase	Reglas del nivel: pawn por defecto (la nave), muestra menu y crea el mapa.
UFachadaJuego	Facade: IniciarPartida / TerminarPartida / MostrarVictoria.
UEstadoJuego (+5 hijas)	State: Menu, Jugando, Pausado, GameOver, Victoria.
AMapaEstelar	Genera por codigo el fondo de estrellas y las paredes del tunel.

Jugador y Armas

Clase	Que hace
ANaveJugador	Pawn del jugador: movimiento, disparo (Object Pool), Subject del Observer, portador del arma (Decorator), habilidades y power-ups.
UArma / UArmaBasica	Componente base del Decorator; el arma basica dispara 1 bala.
UDecoradorArma	Decorator base: envuelve otra arma y delega.
UDecoradorDisparoTriple	Decorator concreto: anade 2 balas en diagonal (abanico).

Enemigos

Clase	Que hace
AEnemigoBase	Padre de los enemigos; contexto del Strategy; recibe dano (con seguro de muerte).
AJefeBase	Padre de los jefes: rebota en vertical y dispara por temporizador (Disparar virtual).
ADragon / APezMonstruo / AKraken	Jefes de nivel 1 / 2 / 3; cada uno con su ataque.
UFabricaEnemigos	Factory: crea enemigos y jefes.

Clase	Que hace
FConstructorEnemigo	Builder: arma un enemigo paso a paso.
AGeneradorEnemigos	Suelta enemigos por temporizador y pide el jefe al limpiar la oleada.
UEstrategiaMovimiento (+hijas)	Strategy: Lineal, ZigZag, Diagonal, Perseguidor.

Proyectiles, Power-ups y UI

Clase	Que hace
AProyectilBase	Bala base; elemento del Object Pool (se enciende/apaga, no se crea/destruye).
AProyectilLaser / LaserVertical / Misil	Variantes: laser rapido perforante, columna vertical, misil ondulante.
ACapsulaPoder	Power-up: al tocar la nave, mejora su arma y da puntos.
UPantallaHUD	Observer: muestra vida, vidas y puntos; se suscribe a la nave.
UPantallaMenu / GameOver / Victoria	Pantallas de UI con sus botones.

7. Decisiones tecnicas y correccion de crashes

Durante el desarrollo se corrigieron varias causas de crash y bugs:

Problema	Solucion
Singleton colgado al cambiar de nivel (puntero a memoria liberada)	Instancia estatica creada en el paquete transitorio + AddToRoot(): vive toda la partida y nunca queda colgada.
Timers con lambda del disparo seguian tras destruir la nave	Se cancelan todos los temporizadores en EndPlay (por handle y por objeto).
El jefe 'subia' 2-3 niveles de golpe al morir	Seguro de muerte (bMuerto) + apagar colision: la logica de muerte corre UNA sola vez.
El Dragon se mataba con sus propias balas	Las balas salen fuera de su cuerpo (las posiciones de disparo ya no caen dentro de su colision).
Contenido de muestra SoulCity (1.5 GB) impedia subir a GitHub	Se ignora en .gitignore (no se usa en el juego).

8. Diccionario de sintaxis (Unreal + C++)

Significado simple de las palabras que aparecen en el código.

A) C++ basico

Palabra / sintaxis	Que significa
<code>class</code>	Define una clase (el molde de los objetos).
<code>public / private / protected</code>	Quien puede usar un miembro: todos / solo la clase / la clase y sus hijas.
<code>void</code>	La funcion no devuelve ningun valor.
<code>virtual</code>	Metodo que las hijas pueden sobrescribir (polimorfismo).
<code>virtual void Funcion()</code>	Funcion sin retorno que se puede sobrescribir.
<code>override</code>	Indica que sobrescribes un metodo virtual del padre.
<code>PURE_VIRTUAL / = 0</code>	Metodo abstracto: las hijas estan obligadas a implementarlo.
<code>static</code>	Pertenece a la CLASE, no a cada objeto; se usa sin instancia.
<code>: public AActor</code>	Herencia: esta clase hereda de AActor.
<code>* (puntero)</code>	Guarda la direccion en memoria de un objeto.
<code>-></code>	Accede a un miembro a traves de un puntero.
<code>::</code>	Operador de ambito (Clase::Metodo).
<code>&</code>	Referencia / 'direccion de'.
<code>nullptr</code>	Puntero vacio (no apunta a nada).
<code>this</code>	Puntero al propio objeto actual.

B) Unreal Engine

Palabra / sintaxis	Que significa
<code>UCLASS()</code>	Registra la clase en el sistema de reflexion de Unreal.
<code>GENERATED_BODY()</code>	Inserta el código que Unreal genera para la clase.
<code>UPROPERTY()</code>	Marca una variable para que Unreal la gestione (GC, editor).
<code>UFUNCTION()</code>	Marca una funcion para Unreal (necesaria para delegados).
<code>UObject / AActor / APawn</code>	Base de todo / base de lo que va en el mundo / actor controlable.
<code>UWorld</code>	El mundo o nivel actual.
<code>FVector / FRotator</code>	Vector 3D (X,Y,Z) / rotacion (Pitch,Yaw,Roll).
<code>TArray<></code>	Lista o arreglo dinamico de Unreal.
<code>TSubclassOf<></code>	Referencia a una CLASE (no a un objeto).
<code>SpawnActor<>()</code>	Crea (instancia) un actor en el mundo.
<code>NewObject<>()</code>	Crea un UObject que no es actor.
<code>Cast<>()</code>	Convierte un puntero a otro tipo si es compatible.
<code>BeginPlay() / EndPlay()</code>	Se ejecuta al entrar / salir el actor del juego.

Palabra / sintaxis	Que significa
<code>Tick(DeltaTime)</code>	Se ejecuta cada frame; DeltaTime = seg. desde el frame anterior.
<code>GetWorld()</code> / <code>Destroy()</code>	Obtiene el mundo / elimina el actor.
<code>FTimerHandle</code> / <code>SetTimer(...)</code>	Identificador / programacion de un temporizador.
<code>AddToRoot()</code>	Protege un UObject del recolector de basura (vive siempre).
<code>TakeDamage(...)</code>	Funcion estandar de Unreal para recibir dano.

C) Delegados / Observer

Palabra / sintaxis	Que significa
<code>DECLARE_DYNAMIC_MULTICAST_DELEGATE...</code>	Declara un 'evento' al que se suscriben varias funciones.
<code>Broadcast(...)</code>	DISPARA el evento: avisa a todos los suscritos (el 'publish').
<code>AddDynamic(obj, &Clase::Func)</code>	SUSCRIBE una funcion al evento (subscribe).
<code>OnComponentBeginOverlap</code> / <code>OnClicked</code>	Eventos integrados de Unreal (solape / clic de boton).

9. Como compilar y ejecutar

- Abrir el proyecto con Unreal Engine 4.27 (doble clic en ImpactoEspacial.uproject).
- Para compilar el código C++: botón **Compile** en el editor, o compilar la solución en Visual Studio.
- Si Visual Studio se queda sin memoria al compilar (error de 'archivo de paginación'), reducir los procesos en paralelo o ampliar la memoria virtual de Windows.
- Jugar con el botón Play del editor.

Carpetas que NO se versionan en git (se regeneran solas): Binaries, Intermediate, Saved, DerivedDataCache, Build, y el contenido de muestra SoulCity.